Поэтапный план реализации ML Inference Service для системы кибербезопасности

Этап 1: MVP — «Работающий скелет»

Цель

Деплоябельный сервис, который принимает запрос на `/predict`, возвращает реальный прогноз от хотя бы одной ML-модели, отдаёт базовую аналитику из БД и имеет минимальный UI для демонстрации. **Всё запускается через `docker-compose up`.**

Что реализуем

Компонент	Детали
Docker Compose	PostgreSQL 15 + FastAPI backend + статический фронт (или mock-UI). Healthcheck для БД
БД: схема + seed	`001_create_tables.sql`, `002_indexes.sql`, загрузка `threats` (227 строк) и `incidents` (2000 строк) через init-скрипты. Таблица `enterprise_profiles` заполняется скриптом канонизации
Backend: скелет FastAPI	`main.py` с CORS, 4 роутера: `health`, `analytics/summary`, `predict`, `threats` (список)
Backend: SQLAlchemy	`session.py`, ORM-модели для `threats`, `incidents`, `enterprise_profiles`. Один repository — `incidents.py` с агрегацией для summary
Backend: ML loader	`ml/loader.py` — загрузка единственного `.pkl` при старте (`incident_classifier`). Fallback на mock если файл отсутствует
Backend: /predict (v0)	Принимает `PredictRequest`, строит feature-вектор (`feature_builder.py` — базовые + temporal фичи), вызывает `incident_classifier`, остальные блоки ответа — **hardcoded/rule-based заглушки** с правдоподобными значениями
ML: Модель 1	`incident_classifier` (CatBoost, бинарная). Обучение на очищенных данных, экспорт `.pkl` + `metadata.json`
ML: Очистка данных	`cleaning.py` — канонизация регионов, типов предприятий, дедупликация. Результат: `incidents_clean.parquet`
ML: Базовые фичи	`base.py` + `temporal.py` — enterprise_type, host_count, region, hour_sin/cos, month_sin/cos, day_of_week, is_weekend, season
Frontend: минимум	Одна страница с формой прогноза (`PredictionForm`) + отображение JSON-ответа в читаемом виде. Можно даже Swagger UI как «фронт»
Pydantic-схемы	`PredictRequest`, `PredictResponse` (полная структура, но часть полей nullable/с дефолтами)
Health endpoint	Проверка БД + наличие модели

Что НЕ делаем

- Модели 2–6 (time, threat, object, vulnerability scoring, recommendations) — заглушки
- SHAP-объяснения — возвращаем пустой массив или mock
- Полноценный React-фронтенд (5 экранов, графики, Recharts)
- Таблица `predictions` (логирование прогнозов)
- Таблица `recommendations`, `demo_scenarios`
- Endpoints: `timeseries`, `regions`, `enterprise-types`, `threats/{code}`, `recommendations`, `scenarios/demo`
- Фильтрация, пагинация, поиск
- CORS для продакшн-доменов
- Логирование запросов, structured logging
- Кэширование, оптимизация запросов
- Форматирование кода (Black/Ruff) — только работоспособность

Временная оценка

Задача	Часы
Docker Compose + БД + init-скрипты	2–3
Очистка данных + базовые фичи	3–4
Обучение incident_classifier + экспорт	2–3
FastAPI скелет + health + analytics/summary	2–3
ML loader + feature_builder + /predict	3–4
Минимальный UI (форма + результат)	2–3
Интеграционное тестирование, фиксы	1–2
Итого	**16–22 часа (~2 дня)**

Ключевые технические решения

1. **Один `.pkl` файл** — не загружаем 6 моделей, только `incident_classifier`. Остальные блоки ответа `/predict` заполняются детерминированно (rule-based заглушки с правдоподобными данными)
2. **Синхронный SQLAlchemy** — не тратим время на async session factory. `create_engine` + `sessionmaker`, dependency injection через `Depends(get_db)`
3. **Feature builder возвращает dict** — не DataFrame. Минимум зависимостей, быстрый старт
4. **Полная структура PredictResponse сразу** — фронтенд и интеграционные тесты работают с финальным контрактом, даже если часть данных — заглушки. Это критично для параллельной разработки
5. **`MODEL_DIR` через env** — backend не знает, откуда модель. Можно подменить на mock

Критерии готовности

docker-compose up --build поднимает 3 контейнера без ошибок

GET /api/v1/health → {"status": "ok", "database": "connected", "models_loaded": 1}

GET /api/v1/analytics/summary → реальные агрегаты из 2000 инцидентов

POST /api/v1/predict с валидным телом → 200 с полной структурой PredictResponse

POST /api/v1/predict с невалидным телом → 422 с описанием ошибки

incident_prediction.probability — реальное значение от CatBoost (не хардкод)

Swagger UI на /docs показывает все реализованные endpoints

Время ответа /predict < 500ms (без оптимизации, но работает)

Этап 2: Production-Ready — «Полный функционал»

Цель

Все 11 endpoints работают с реальными данными. Все 6 моделей (или их rule-based эквиваленты) интегрированы. SHAP-объяснения генерируются. Полноценный React-фронтенд с 5 экранами. Логирование прогнозов в БД. Демо-сценарии. Система готова к демонстрации на защите/хакатоне.

Что реализуем

Компонент	Детали
ML: Модели 2a, 2b	`attack_time_bucket` (CatBoost multiclass) + `attack_time_hour` (CatBoost regressor). Обучение, экспорт
ML: Модель 3	`threat_type` (LightGBM multiclass). LabelEncoder'ы, top-3 предсказания, экспорт encoders
ML: Модель 4	`target_object` — rule-based маппинг `threat_code → impact_object → category`. Словарь `OBJECT_MAPPING`
ML: Модель 5	`vulnerability` — scoring formula (6 факторов, экспертные веса). `calculate_vulnerability_score()`
ML: Модель 6	`recommendation` — rule engine. `select_recommendations()` + `generate_time_recommendation()`
ML: SHAP	`TreeExplainer` для incident_classifier. `get_shap_explanation()` → top-5 фичей с текстовыми объяснениями
ML: Агрегированные фичи	`aggregations.py` — incidents_by_region, success_rate_by_type, threat_frequency, days_since_last_incident
Backend: Полный /predict	Оркестрация всех 6 моделей в `prediction_service.py`. Реальные данные во всех блоках ответа
Backend: Все analytics	`timeseries`, `regions`, `enterprise-types` — реальные SQL-агрегации с фильтрами
Backend: Threats	Список с пагинацией + поиском, детальная карточка `threats/{ubi_code}` с incident_stats и related_recommendations
Backend: Recommendations	CRUD из БД с фильтрацией по threat_code, vuln_level, target_object
Backend: Demo scenarios	Таблица `demo scenarios` заполнена 3 сценариями. `GET /scenarios/demo` + `POST /scenarios/demo/{id}/run`
Backend: Логирование прогнозов	Каждый вызов `/predict` сохраняется в таблицу `predictions` с features_json и explanation_json

Компонент	Детали
Backend: Recommendation engine	`recommendation_service.py` — подбор рекомендаций по контексту (threat + object + vuln_level + time)
Backend: Обогащение ответа	threat_name, impact_object, recommendation details — всё из БД, не хардкод
Backend: Error handling	Единый формат ошибок (ЧАСТЬ X документа). Exception handlers для 400/404/422/500/503
Backend: Structured logging	Каждый запрос логируется: endpoint, duration_ms, status_code. Ошибки — с traceback
Frontend: 5 экранов	Dashboard, Prediction, Vulnerability, Recommendations, ThreatCatalog — полная реализация
Frontend: Графики	Recharts: LineChart (timeseries), BarChart (regions, enterprise-types), PieChart (threats), ShapWaterfall
Frontend: Компоненты	KpiCard, RiskCard, ThreatCard, RecommendationCard, PredictionForm, FilterPanel
Frontend: States	Loading spinner, Error message, Empty state на каждом экране
Frontend: Демо-кнопки	3 кнопки на PredictionPage — заполняют форму из `/scenarios/demo` и автоматически запускают прогноз
Frontend: TypeScript типы	Полная типизация всех API-ответов (types/*.ts)
Таблица recommendations	Заполнена ~50 записями (по 2–3 на топовые угрозы)
Таблица demo_scenarios	3 записи: critical (Медицина/Якутия/1500), medium (НКО/Москва/120), low (Образование/Краснодар/45)
Business impact	Формула расчёта ущерба по отрасли и host_count
Pydantic: полные схемы	Все response-модели из ЧАСТИ IV.4

Что НЕ делаем

- SHAP-кэш (каждый запрос пересчитывает SHAP — допустимо при <300ms)
- Data drift мониторинг (нет сравнения распределений train vs inference)
- A/B тестирование моделей (одна версия — v1)
- Async SQLAlchemy (синхронный — достаточно для одного пользователя)
- Redis/Memcached кэширование
- Rate limiting
- Аутентификация/авторизация
- CI/CD pipeline
- Unit-тесты (только ручное интеграционное тестирование)
- Мониторинг (Prometheus/Grafana)
- Model registry (MLflow)
- Оптимизация SQL-запросов (EXPLAIN ANALYZE)
- Batch prediction
- WebSocket для real-time обновлений

Временная оценка

Задача	Часы
ML: Модели 2a + 2b (time)	4–5

Задача	Часы
ML: Модель 3 (threat type) + encoders	4–5
ML: Модели 4–6 (rule-based)	3–4
ML: SHAP explainer + текстовые объяснения	4–5
ML: Агрегированные фичи + переобучение модели 1	2–3
Backend: prediction_service (оркестрация 6 моделей)	4–6
Backend: analytics endpoints (4 штуки) + фильтры	4–5
Backend: threats (список + детали) + recommendations	3–4
Backend: demo scenarios + run	2–3
Backend: predictions logging + error handling + logging	3–4
Backend: recommendation_service + business_impact	2–3
Frontend: Layout + Router + API client	3–4
Frontend: Dashboard (4 графика + KPI + фильтры)	6–8
Frontend: PredictionPage (форма + все блоки результата)	6–8
Frontend: VulnerabilityPage + RecommendationsPage + ThreatCatalog	6–8
Frontend: SHAP waterfall + демо-кнопки + states	3–4
Seed data: recommendations (50 записей) + demo_scenarios (3)	2–3
Интеграционное тестирование end-to-end	4–6
Полировка: адаптивность, Toast, форматирование	3–4
Итого	**68–90 часов (~8–12 рабочих дней)**

Ключевые технические решения

1. **Prediction Service как оркестратор** — единый класс `PredictionService` вызывает все 6 моделей последовательно, каждая модель изолирована. Если одна падает — fallback на заглушку, остальные работают. Паттерн: graceful degradation
2. **SHAP вычисляется синхронно** — при 2000 строках обучения TreeExplainer работает за 20–50ms. Кэш не нужен. Если будет медленно — вынесем в Этап 3
3. **LabelEncoder'ы сериализуются вместе с моделью** — `threat_type_encoders.pkl` содержит все 4 энкодера. Backend загружает их при старте и использует для transform входных данных
4. **Recommendations — гибридный подход** — БД содержит справочник, rule engine фильтрует и ранжирует. Контекстная рекомендация по времени генерируется динамически
5. **Frontend: RTK Query** — автоматическое кэширование, invalidation, loading states. Один `api.ts` с `createApi`, слайсы для каждого endpoint
6. **Единый error handler** — FastAPI exception handlers для `RequestValidationError`, `HTTPException`, `MLInferenceError` (кастомный). Все ответы — по формату из ЧАСТИ X
7. **Feature builder v2** — принимает `PredictRequest` + данные из БД (агрегаты по региону/отрасли), возвращает полный feature-вектор для всех моделей

Критерии готовности

- [ ] Все 11 endpoints возвращают корректные ответы (проверка через Swagger)
- [ ] POST /predict → все 6 блоков ответа содержат реальные данные (не заглушки)

SHAP explanations.top_features содержит 5 фичей с impact > 0

Рекомендации привязаны к predicted_threat (не хардкод)

vulnerability_assessment.score коррелирует с incident_prediction.probability

Dashboard: 4 KPI-карточки + 4 графика отображают реальные данные из БД

Dashboard: фильтры по региону и отрасли работают

PredictionPage: форма → submit → результат отображается корректно

PredictionPage: SHAP waterfall отрисовывается

PredictionPage: 3 демо-кнопки заполняют форму и запускают прогноз

ThreatCatalog: поиск + пагинация работают

Threats/{code}: карточка с incident_stats и related_recommendations

RecommendationsPage: фильтрация по threat_code и vuln_level

VulnerabilityPage: таблица предприятий с risk_score

Каждый прогноз сохраняется в таблицу predictions (проверка через SQL)

Ошибки отображаются как Toast, а не белый экран

GET /health → models_loaded: 4 (incident, time_bucket, time_hour, threat_type)

Время ответа /predict < 300ms

Время ответа /analytics/* < 100ms

docker-compose up --build на чистой машине — всё работает

README.md с инструкцией запуска

Этап 3: Полная оптимизация — «Production-grade»

Цель

Система готова к реальной эксплуатации: SHAP-кэш для повторных запросов, мониторинг data drift, A/B-тестирование моделей, async I/O, кэширование, observability, тесты, CI/CD. Масштабируемость и надёжность.

Что реализуем

Компонент	Детали

Компонент	Детали
SHAP-кэш	Redis-кэш для SHAP-объяснений. Ключ: hash(features_dict). TTL: 1 час. При повторном запросе с теми же параметрами — мгновенный ответ. Снижение latency /predict с ~200ms до ~50ms для cache hit
Data Drift мониторинг	При каждом вызове `/predict` — сохранение feature-вектора. Фоновая задача (Celery/APScheduler): раз в час сравнение распределений inference vs training (PSI, KS-test). Алерт если PSI > 0.2
A/B тестирование моделей	Поддержка нескольких версий моделей (`models/v1/` , `models/v2/`). Header `X-Model-Version` или конфиг-флаг. Логирование версии в `predictions.model_version` . Dashboard сравнения метрик v1 vs v2
Async SQLAlchemy	Миграция на `asyncpg` + `AsyncSession` . Все repository-методы — `async def` . Реальный выигрыш при concurrent requests
Redis	Кэширование: SHAP-результатов, analytics/summary (TTL 5 min), threats list (TTL 30 min). Session store для будущей авторизации
Rate Limiting	`slowapi` или кастомный middleware. /predict: 10 req/min per IP. /analytics: 60 req/min
Batch Prediction	`POST /api/v1/predict/batch` — принимает массив до 100 предприятий. Параллельный inference. Ответ: массив PredictResponse
Model Registry	MLflow tracking server. Логирование метрик, параметров, артефактов при обучении. Загрузка моделей из registry вместо файловой системы
Observability	Prometheus metrics (request_duration, model_inference_time, cache_hit_rate, prediction_distribution). Grafana dashboards. Structured JSON logging (structlog)
CI/CD	GitHub Actions: lint (ruff) → type-check (mypy) → unit tests (pytest) → integration tests → build Docker → push to registry
Unit тесты	pytest: repositories (mock DB), services (mock models), API endpoints (TestClient). Coverage > 80%
Integration тесты	pytest + testcontainers: реальный PostgreSQL, реальные модели. Проверка end-to-end flow
Аутентификация	JWT tokens. `/auth/login` → token. Middleware проверяет token на protected endpoints. Роли: admin, analyst, viewer
Оптимизация SQL	EXPLAIN ANALYZE для всех analytics-запросов. Materialized views для тяжёлых агрегаций. Partial indexes
Model warm-up	При старте backend — прогон dummy prediction через все модели (JIT-компиляция, загрузка в кэш CPU). Первый реальный запрос — быстрый
SHAP background computation	Для новых уникальных feature-комбинаций — асинхронный расчёт SHAP в background worker. Клиент получает prediction сразу, SHAP — через polling или WebSocket
Drift alerting	Webhook/email при обнаружении drift. Автоматическое переключение на fallback-модель если drift критический
Feature Store	Предрасчитанные агрегаты (incidents_by_region, success_rate_by_type) обновляются по расписанию, не при каждом запросе
API versioning	`/api/v1/` и `/api/v2/` — параллельная поддержка. v2 может иметь другую структуру ответа
Graceful shutdown	SIGTERM → завершение текущих запросов → закрытие DB connections → exit. Kubernetes-ready
Helm chart	Kubernetes deployment: backend (2 replicas), PostgreSQL (StatefulSet), Redis (single), frontend (2 replicas), Ingress

Что это завершает (ничего не откладываем)

Этап 3 — финальный. После него система полностью соответствует production-grade стандартам для ML-сервиса в кибербезопасности.

Временная оценка

Задача	Часы
Redis интеграция + SHAP-кэш	6–8
Async SQLAlchemy миграция	8–12
Data Drift мониторинг (PSI, KS-test, alerting)	12–16
A/B тестирование (multi-version loader, routing, comparison)	10–14
Batch prediction endpoint	4–6
MLflow model registry	8–12
Prometheus + Grafana (metrics, dashboards)	8–10
Structured logging (structlog)	3–4
CI/CD pipeline (GitHub Actions)	6–8
Unit тесты (repositories, services, endpoints)	12–16
Integration тесты (testcontainers)	8–10
Аутентификация (JWT + roles)	8–10
SQL оптимизация (materialized views, partial indexes)	4–6
Model warm-up + SHAP background worker	6–8
Feature Store (предрасчитанные агрегаты)	6–8
Rate limiting + API versioning	4–6
Graceful shutdown + Helm chart	8–12
Документация (API docs, runbook, architecture decision records)	6–8
Нагрузочное тестирование (locust) + оптимизация по результатам	6–8
Итого	**130–180 часов (~16–22 рабочих дня)**

Ключевые технические решения

1. **SHAP-кэш: Redis с content-addressable ключами**

   import hashlib, json

   def shap_cache_key(features: dict) -> str:

   ```
   canonical \= json.dumps(features, sort\_keys=True)

   return f"shap:{hashlib.sha256(canonical.encode()).hexdigest()\[:16\]}"
   ```

   TTL = 3600s. Cache hit rate ожидается ~40% (повторные запросы с теми же параметрами предприятия).

2. **Data Drift: Population Stability Index (PSI)**

   def calculate_psi(expected, actual, bins=10):

   ```
   \# Разбиваем на бины по квантилям training distribution

   \# PSI \< 0.1 → нет drift, 0.1-0.2 → moderate, \>0.2 → significant
   ```

   Мониторим top-5 фичей по SHAP importance. Drift по любой из них → алерт.

3. **A/B: Header-based routing**

   @router.post("/predict")

   async def predict(request: PredictRequest,

```
              x\_model\_version: str \= Header("v1")):

model\_set \= model\_registry.get(x\_model\_version)

\# ... inference с выбранной версией
```

   Логирование в `predictions.model_version` позволяет сравнивать метрики post-hoc.

4. **Async pipeline с fallback**

   async def run_inference(request):

```
results \= await asyncio.gather(

    run\_incident\_model(request),

    run\_time\_model(request),

    run\_threat\_model(request),

    return\_exceptions=True  \# не падаем если одна модель сломалась

)

\# Для каждого результата: если Exception → fallback
```

5. **Feature Store: materialized view + cron refresh**

   CREATE MATERIALIZED VIEW feature_aggregates AS

   SELECT region, enterprise_type,

```
   COUNT(\*) as incident\_count,

   AVG(success::int) as success\_rate,

   AVG(host\_count) as avg\_hosts
```

   FROM incidents GROUP BY region, enterprise_type;

   -- Refresh каждые 5 минут

   REFRESH MATERIALIZED VIEW CONCURRENTLY feature_aggregates;

6. **Observability: RED metrics**

- **R**ate: requests/sec per endpoint
- **E**rrors: error rate per endpoint
- **D**uration: p50, p95, p99 latency
- Плюс ML-специфичные: inference_time_ms, shap_compute_time_ms, cache_hit_rate, drift_psi

Критерии готовности

SHAP-кэш: повторный запрос с теми же параметрами → inference_time_ms < 50ms

SHAP-кэш: cache hit rate > 30% на реалистичной нагрузке

Data Drift: PSI вычисляется для top-5 фичей, результат доступен через /api/v1/monitoring/drift

Data Drift: алерт срабатывает при PSI > 0.2 (проверка с синтетическим drift)

A/B: запрос с X-Model-Version: v2 использует модели из models/v2/

A/B: /api/v1/monitoring/ab-comparison → метрики v1 vs v2

Async: все DB-запросы — async (проверка: нет sync session в коде)

Batch: POST /predict/batch с 50 предприятиями → ответ < 3 секунд

Redis: analytics/summary кэшируется (проверка: второй запрос < 5ms)

MLflow: модели загружаются из registry (не из файловой системы)

Prometheus: /metrics endpoint отдаёт request_duration_seconds, model_inference_seconds

Grafana: dashboard с 4 панелями (rate, errors, duration, ML metrics)

CI: push в main → lint + tests + build проходят за < 5 минут

Unit тесты: coverage > 80% (pytest --cov)

Integration тесты: 10+ тестов проходят с реальной БД (testcontainers)

Auth: /predict без токена → 401. С токеном → 200

Rate limit: 11-й запрос за минуту → 429

SQL: EXPLAIN ANALYZE для analytics/summary → Seq Scan отсутствует

Warm-up: первый реальный запрос после старта → < 300ms (не cold start)

Нагрузочный тест: 50 concurrent users, p95 < 500ms, error rate < 1%

Helm: kubectl apply → система поднимается в Kubernetes

Graceful shutdown: SIGTERM → текущие запросы завершаются, новые отклоняются

Документация: ADR для каждого ключевого решения, runbook для on-call

---

Сводная таблица сравнения этапов

Характеристика	Этап 1: MVP	Этап 2: Production-Ready	Этап 3: Full Optimization
Цель	Работающий прототип	Полный функционал для демо	Production-grade система
Срок	16–22 часа (1–2 дня)	68–90 часов (8–12 дней)	130–180 часов (16–22 дня)
Cumulative	1–2 дня	10–14 дней	26–36 дней
ML-модели	1 (incident_classifier)	4 реальных + 2 rule-based	4 реальных + 2 rule-based + v2
SHAP	Mock/пустой массив	Реальный, синхронный	Реальный + Redis-кэш
Endpoints	4 из 11	11 из 11	11 + batch + monitoring + auth
Frontend	Минимальная форма	5 полных экранов + графики	5 экранов + admin panel
БД	3 таблицы (threats, incidents, profiles)	6 таблиц	6 таблиц + materialized views
Кэширование	Нет	Нет	Redis (SHAP, analytics)
SQLAlchemy	Синхронный	Синхронный	Async (asyncpg)
Мониторинг	Health endpoint	Health + structured logs	Prometheus + Grafana + drift
Data Drift	Нет	Нет	PSI + KS-test + alerting
A/B тесты	Нет	Нет	Header-based routing + comparison
Тесты	Ручное тестирование	Ручное + Swagger	Unit + Integration + Load
CI/CD	Нет	Нет	GitHub Actions
Auth	Нет	Нет	JWT + roles
Rate Limiting	Нет	Нет	slowapi
Batch	Нет	Нет	POST /predict/batch
Model Registry	Файловая система	Файловая система	MLflow
Deploy	docker-compose	docker-compose	docker-compose + Helm
SLA /predict	< 500ms (best effort)	< 300ms	< 300ms (p95), < 50ms cache hit
Отказоустойчивость	Падает при ошибке модели	Fallback на заглушки	Graceful degradation + retry
Документация	README с запуском	README + Swagger + DATA_CLEANING	README + ADR + Runbook + API docs